

Оглавление

Оглавление

1. Цель и главный результат

2. Математическая модель маски

3. Оптимизация № 1: один канал вместо четырёх

4. Оптимизация № 2: зеркало без отдельного хранения

5. Оптимизация № 3: аффинный переход между периодами

5.1. Универсальный декодер

5.2. Псевдокод

6. Алгоритм генерации полной и очищенной масок

6.1. Полная маска G_p

6.2. Очищенная маска E_p

6.3. Генерация чисел напрямую

7. Тессеракт и группа B_4 : пакет из двух масок

8. 24-ячейник и группа F_4 : пакет из шести масок

8.1. Три слоя F_4

8.2. Добавление двоичного слоя

8.3. Один 24-ячейник

9. 600-ячейник: пакет из тридцати масок

9.1. Точная групповая цепочка

9.2. Иерархия состояний

9.3. Что здесь вычисляется

10. E_8 и политоп Гоше 4_{21} : пакет из шестидесяти масок

10.1. Два H_4 -слоя

10.2. Иерархическая координата

10.3. Результат

11. Роль золотого сечения, дуальности и расслоения Хопфа

11.1. Золотое сечение

11.2. Что здесь называется дуалом

11.3. Расслоение Хопфа

12. Точные коэффициенты оптимизации

12.1. Структурные коэффициенты

12.2. Почему общий результат не равен $4 \cdot 2 \cdot 6 \cdot 30 \cdot 60$

12.3. Сравнение с плотными битовыми половинами

12.4. Аффинный автомат не ограничен числом 60

13. Что именно хранится

13.1. Глобальные константы формата

13.2. Seed макроблока

13.3. Prime-only режим

14. Алгоритмы и проверочные программы

14.1. Арифметический аффинный автомат

14.2. Точная групповая проверка B_4/F_4

14.3. Алгоритм 600-ячейникового блока

14.4. Алгоритм E_8 -блока

15. Рекомендуемая архитектура

15.1. Математическое ядро

15.2. Локальный уровень

15.3. Средний уровень

15.4. Верхний уровень

15.5. Практическая рекомендация

15.6. Оптимальный формат исполнительного слоя: хранить как E_8 , исполнять как 24-ячейники

Сравнение selector-форматов

Горячий цикл

Зачем сохранять 24-ячейниковое деление

15.7. Какие предложения из `optimizations.md` использованы

15.8. Параллельная архитектура CPU + GPU

Конвейер

15.9. Память CPU + GPU

16. Текущее состояние реализации и измеренные тесты производительности

16.1. CPU

16.2. GPU

16.3. Устойчивый гибридный режим

16.4. Память

17. Практический рубеж: первые 10^{12} простых

17.1. Он ещё реалистичен на одной машине

17.2. Он проверяет всю инфраструктуру, а не только одну формулу

17.3. Полная выгрузка не нужна как первый режим

17.4. Сохраняемое состояние, продолжение и зафиксированный сегмент

17.5. Практические команды для фонового прогона

18. Где оптимизации реально дают выигрыш

18.1. Один канал вместо четырёх

18.2. Зеркало без отдельного хранения

18.3. `E8Bitmap`, `SparseCSR`, `ImplicitFull`

18.4. CPU, GPU и гибридный режим

18.5. Почему LUT не нужен в горячем пути CPU

18.6. Почему постоянный пул CUDA обязателен

19. Почему диапазон $10^{19} \rightarrow 10^{20}$ не является первой практической целью

20. Большие отдельные простые: `isPrime(N)` и `nextPrime(N)`

20.1. Что уже измерено

20.2. Текущая поверхность запросов

20.3. Чем локальный запрос отличается от полного префиксного прохода

20.4. Ограничение версии для больших целых

20.5. Маска как доказательство и маска как фильтр

20.6. Быстрый `isPrime(N)`

20.7. Уточнённый `isPrime(N)`

20.8. Быстрый `nextPrime(N)`

20.9. Уточнённый `nextPrime(N)`

20.10. Оценка времени для 1000-значных чисел

20.11. Роль CPU/GPU оптимизаций

20.12. Прогресс вычислений

20.13. ЕСРР не является ML/тригонометрическим вычислением

20.14. Команды и статусы

20.15. Итоговая семантика слоя больших целых

21. Практические сценарии применения

21.1. Последовательный префиксный поиск

21.2. Проверка конкретного большого числа

21.3. Поиск первого простого после N

21.4. Гибридный путь

21.5. Кластер и удалённые диапазоны

22. Источники

22.1. Внутренние ссылки на текущие расчёты

22.2. Внешние математические источники

23. Практический запуск длинного прогона через Makefile

23.1. Wheel210-only режим хранения

24. Приложение: текущий исполнительный и операционный слой

25. Универсальные операции Makefile

26. Модель истины для длинных прогонов

26.1. Утверждение текущего прогона

26.2. Зафиксированная истина архива

26.3. Буферизированная истина в RAM

27. Архивные запросы к простым числам

28. Текущая рабочая форма исполнительного слоя

29. Цели и нейтральная дорожная карта

30. Локальный интерфейс для `isPrime(N)` и `nextPrime(N)`

30.1. Что показывает интерфейс

30.2. Локальная HTTP-поверхность

30.3. Почему интерфейс выглядит именно так

30.4. Связь интерфейса с архивом

30.5. Локальный запуск

Быстрое вычисление простых чисел

Групповое кодирование циклических масок, исполнительный слой E8/24-cell и запросы к большим целым числам.

Оглавление

1. [Цель и главный результат](#)
2. [Математическая модель маски](#)
3. [Оптимизация № 1: один канал вместо четырёх](#)
4. [Оптимизация № 2: зеркало без отдельного хранения](#)
5. [Оптимизация № 3: аффинный переход между периодами](#)
6. [Алгоритм генерации полной и очищенной масок](#)
7. [Тессеракт и группа \$B_4\$: пакет из двух масок](#)
8. [24-ячейник и группа \$F_4\$: пакет из шести масок](#)
9. [600-ячейник: пакет из тридцати масок](#)
10. [\$E_8\$ и политоп Гоше \$4_{21}\$: пакет из шестидесяти масок](#)
11. [Роль золотого сечения, дуальности и расслоения Хопфа](#)
12. [Точные коэффициенты оптимизации](#)
13. [Что именно хранится](#)
14. [Алгоритмы и проверочные программы](#)
15. [Рекомендуемая архитектура](#)
16. [Текущее состояние реализации и измеренные тесты производительности](#)
17. [Практический рубеж: первые \$10^{12}\$ простых](#)
18. [Где оптимизации реально дают выигрыш](#)
19. [Почему диапазон \$10^{19} \rightarrow 10^{20}\$ не является первой практической целью](#)
20. [Большие отдельные простые: `isPrime\(N\)` и `nextPrime\(N\)`](#)
21. [Практические сценарии применения](#)
22. [Источники](#)
23. [Практический запуск длинного прогона через Makefile](#)
24. [Приложение: текущий исполнительный и операционный слой](#)
25. [Универсальные операции Makefile](#)
26. [Модель истины для длинных прогонов](#)
27. [Архивные запросы к простым числам](#)
28. [Текущая рабочая форма исполнительного слоя](#)
29. [Цели и нейтральная дорожная карта](#)
30. [Локальный интерфейс для `isPrime\(N\)` и `nextPrime\(N\)`](#)

1. Цель и главный результат

Маска рассматривается как первичный циклический объект, из которого порождаются числа. Плотная строка битов не является обязательным форматом хранения.

Для каждого периода требуется знать:

- начальное условие;
- общий канальный генератор;
- правило перехода между соседними периодами;
- длину вычисляемого диапазона.

Найдена следующая иерархия точных пакетных представлений:

Геометрия	Число состояний	Каналов	Масок в пакете	Пакетное сокращение seed
Канальный цикл C_4	4	4	1	$1\times$
Тессерактный слой B_4	8	4	2	$2\times$
Один 24-ячейник	24	4	6	$6\times$
Один 600-ячейник	120	4	30	$30\times$
Корни E_8 , политоп 4_{21}	240	4	60	$60\times$

Последняя строка означает:

Одно начальное условие E_8 -макроблока вместе с общими операторами порождает 60 последовательных масок и все четыре канала каждой маски.

Относительно уже построенного 24-ячейникового пакета это даёт потенциальное укрупнение:

$$\frac{60}{6} = 10\times.$$

Относительно независимого хранения четырёх канальных seed каждой маски один E_8 -макроблок охватывает:

$$60 \cdot 4 = 240$$

канальных состояний.

При этом число вершин многогранника само по себе ничего не сжимает. Сжатие возникает только потому, что состояния вычисляются из:

1. общего seed;
2. фиксированных групповых генераторов;
3. точного аффинного кокцикла.

2. Математическая модель маски

Пусть:

$$p = 10q + r, \quad r \in \{1, 3, 7, 9\}.$$

Четыре канала задаются множителями:

$$m_a \in (1, 3, 9, 7), \quad a \in \mathbb{Z}/4\mathbb{Z}.$$

Событие канала a имеет числовое значение:

$$x_{q,r,a} = m_a p.$$

Разложим его по основанию 10:

$$x_{q,r,a} = 10k_{q,r,a} + d_{q,r,a}.$$

Тогда:

- $k_{q,r,a}$ — позиция в маске;
- $d_{q,r,a} \in \{1, 3, 7, 9\}$ — десятичный канал;
- четыре значения a задают четыре специальные позиции периода.

Полная маска G_p содержит все четыре события:

$$p, \quad 3p, \quad 9p, \quad 7p.$$

Очищенная маска E_p отличается удалением самопопадания:

$$p.$$

3. Оптимизация № 1: один канал вместо четырёх

Определим канальный оператор:

$$T_p(x) = 3x \bmod 10p.$$

Он создаёт цикл:

$$1 \longrightarrow 3 \longrightarrow 9 \longrightarrow 7 \longrightarrow 1.$$

Следовательно:

$$\langle T_p \rangle \cong C_4.$$

Из одного события $x = p$ вычисляются:

$$T_p^0(p) = p,$$

$$T_p^1(p) = 3p,$$

$$T_p^2(p) = 9p,$$

$$T_p^3(p) = 7p.$$

Поэтому четыре канала не должны храниться независимо.

Если плотное четырёхканальное представление одного периода занимает:

$$B_{\text{full}}(p) = 4p$$

бит, то один реализованный базовый канал занимает:

$$B_{\text{channel}}(p) = p$$

бит.

Первое точное сокращение:

$$4\times.$$

На практике полный базовый канал также не требуется материализовать: его события вычисляются непосредственно из seed.

4. Оптимизация № 2: зеркало без отдельного хранения

Зеркальное преобразование действует так:

$$R_p(k, d) = (p - 1 - k, 10 - d).$$

Оно меняет пары:

$$1 \leftrightarrow 9, \quad 3 \leftrightarrow 7.$$

На событиях маски выполняется:

$$R_p = T_p^2.$$

Действительно:

$$T_p^2(x) = 9x \equiv -x \pmod{10p},$$

поскольку $p \mid x$.

Зеркало уже находится внутри канальной группы:

$$\langle R_p \rangle = \langle T_p^2 \rangle \subset C_4.$$

Поэтому зеркало не требует:

- второй половины маски;
- отдельного генератора;
- отдельного бита направления;
- отдельного множителя $2\times$ поверх канального $4\times$.

Иначе одно и то же преобразование было бы учтено дважды.

5. Оптимизация № 3: аффинный переход между периодами

Рассмотрим соседний период того же десятичного класса:

$$p(q+1, r) = p(q, r) + 10.$$

Для канала a :

$$x_{q+1, r, a} = m_a(p+10) = x_{q, r, a} + 10m_a.$$

Следовательно:

$$k_{q+1, r, a} = k_{q, r, a} + m_a$$

и:

$$d_{q+1, r, a} = d_{q, r, a}.$$

Позиции четырёх каналов сдвигаются на:

(1, 3, 9, 7).

Через n периодов:

$$k_{q+n,r,a} = k_{q,r,a} + nm_a.$$

Это основной межмасочный алгоритм.

Он справедлив для любого:

$$n \geq 0.$$

Следовательно, пакеты из 2, 3, 6, 30 и 60 масок — не разные арифметические алгоритмы. Это разные геометрические размеры блока одного универсального аффинного автомата.

5.1. Универсальный декодер

Для базового индекса q_0 , номера маски n и канала a :

$$p_n = 10(q_0 + n) + r,$$

$$x_{n,a} = m_a p_n,$$

$$(k_{n,a}, d_{n,a}) = \text{divmod}(x_{n,a}, 10).$$

Эквивалентная рекуррентная форма:

$$k_{n+1,a} = k_{n,a} + m_a.$$

5.2. Псевдокод

```
input:
    q0      начальный индекс
    r       один из 1, 3, 7, 9
    count   число масок

multipliers = [1, 3, 9, 7]

for n in 0 .. count-1:
    p = 10 * (q0 + n) + r

    for a in 0 .. 3:
        x = multipliers[a] * p
```

```
k, d = divmod(x, 10)
emit(period=p, channel=a, position=k, digit=d)
```

Плотная маска при этом не строится.

6. Алгоритм генерации полной и очищенной масок

6.1. Полная маска G_p

Для каждого периода p создаются четыре события:

$$\mathcal{G}_p = \{p, 3p, 9p, 7p\}.$$

Каждое событие преобразуется в координату:

$$(k, d) = \text{divmod}(x, 10).$$

Если требуется строка длины p , все позиции сначала получают символ `f`, после чего в четырёх вычисленных позициях очищается соответствующий бит.

6.2. Очищенная маска E_p

Для маски E_p не создаётся событие:

$$x = p.$$

То есть используется проектор:

$$P_E : \mathcal{G}_p \mapsto \mathcal{G}_p \setminus \{p\}.$$

Правило очистки одинаково для всех периодов и не является индивидуальными данными маски.

6.3. Генерация чисел напрямую

Если конечная задача состоит в генерации или вычёркивании чисел, строковое представление `E/G` не требуется вообще.

Автомат выдаёт непосредственно:

$$x_{n,a} + 10p_n t, \quad t = 0, 1, 2, \dots$$

для нужных каналов и периодов.

7. Тессеракт и группа B_4 : пакет из двух масок

В группе симметрий тессеракта $W(B_4)$ используется подгруппа:

$$H_2 \cong C_4 \times C_2.$$

Здесь:

- C_4 — четыре канала;
- C_2 — два последовательных значения $q \bmod 2$.

Состояние:

$$(a, b) \in C_4 \times C_2$$

задаёт:

$$2 \text{ маски} \times 4 \text{ канала} = 8$$

событий.

Позиционный переход между масками:

$$k \longmapsto k + m_a.$$

Точное пакетное сокращение числа независимых seed:

$$2 \times.$$

Вершины и ориентированные рёбра тессеракта не хранятся. Они вычисляются как состояния действия группы.

8. 24-ячейник и группа F_4 : пакет из шести масок

8.1. Три слоя F_4

Пусть $c \in W(F_4)$ — элемент Кокстера порядка 12:

$$c^{12} = 1.$$

Определим:

$$A = c^3, \quad L = c^4.$$

Тогда:

$$|A| = 4, \quad |L| = 3, \quad AL = LA.$$

Поэтому:

$$\langle A, L \rangle \cong C_4 \times C_3 \cong C_{12}.$$

Это кодирует:

$$3 \text{ маски} \times 4 \text{ канала} = 12$$

состояний.

8.2. Добавление двоичного слоя

Совместная структура:

$$H_6 \cong C_4 \times C_2 \times C_3$$

имеет порядок:

$$|H_6| = 24.$$

Номер маски:

$$n \in \mathbb{Z}/6\mathbb{Z}$$

восстанавливается из:

$$b = n \bmod 2, \quad \ell = n \bmod 3$$

по китайской теореме об остатках:

$$n \equiv 3b + 4\ell \pmod{6}.$$

Состояние:

$$(a, b, \ell) \in C_4 \times C_2 \times C_3$$

кодирует:

$$6 \text{ масок} \times 4 \text{ канала} = 24$$

события.

8.3. Один 24-ячейник

На 24 вершинах реализуется транзитивное скрученное действие порядка 24.

Эквивалентный групповой язык использует бинарную тетраэдральную группу:

$$2T, \quad |2T| = 24.$$

В ней выбирается канальная подгруппа:

$$C_4 \subset 2T.$$

Число её левых классов:

$$[2T : C_4] = \frac{24}{4} = 6.$$

Поэтому множество состояний имеет точную факторизацию:

$$2T = \bigsqcup_{j=0}^5 t_j C_4.$$

Здесь:

- номер класса j — номер одной из шести масок;
- элемент C_4 — номер канала.

Точное пакетное сокращение seed:

$$6 \times.$$

Относительно независимых канальных seed пакет содержит:

$$6 \cdot 4 = 24$$

вычисляемых состояний.

9. 600-ячейник: пакет из тридцати масок

9.1. Точная групповая цепочка

120 вершин 600-ячейника образуют бинарную икосаэдральную группу:

$$2I, \quad |2I| = 120.$$

В неё входит бинарная тетраэдральная группа:

$$2T \subset 2I.$$

Индекс:

$$[2I : 2T] = \frac{120}{24} = 5.$$

Следовательно:

$$2I = \bigsqcup_{i=0}^4 g_i 2T.$$

Каждый класс $g_i 2T$ является 24-ячейником.

В статье [The Geometry of \$H_4\$ Polytopes](#) доказано:

- в 600-ячейнике находятся 25 различных 24-ячейников;
- существует ровно 10 разбиений 120 вершин;
- каждое такое разбиение состоит из пяти непересекающихся 24-ячейников.

Для кодирования достаточно выбрать одно из этих разбиений как глобальный инвариант формата.

9.2. Иерархия состояний

Используем цепочку:

$$C_4 \subset 2T \subset 2I.$$

Тогда:

$$|2I| = [2I : 2T] [2T : C_4] |C_4|.$$

То есть:

$$120 = 5 \cdot 6 \cdot 4.$$

Координата состояния:

$$(i, j, a) \in \{0, \dots, 4\} \times \{0, \dots, 5\} \times \mathbb{Z}/4\mathbb{Z}.$$

Она означает:

- i — один из пяти 24-ячейников;
- j — одна из шести масок внутри 24-ячейника;
- a — один из четырёх каналов.

Номер маски:

$$n = 6i + j, \quad 0 \leq n < 30.$$

Числовой декодер:

$$p_n = 10(q_0 + n) + r,$$

$$x_{n,a} = m_a p_n.$$

Следовательно, один 600-ячейниковый макроблок кодирует:

$$30 \text{ масок} \times 4 \text{ канала}.$$

Пакетное сокращение числа независимых seed:

$$30 \times.$$

Относительно 24-ячейникового пакета:

$$5 \times.$$

9.3. Что здесь вычисляется

Геометрический адрес состояния:

$$g_i t_j c^a$$

вычисляется из:

- фиксированного трансверсала $\{g_i\}$ группы $2T$ в $2I$;
- фиксированного трансверсала $\{t_j\}$ группы C_4 в $2T$;
- канального генератора c порядка 4.

Эти таблицы и генераторы являются общими для всех макроблоков и не дублируются для каждой маски.

10. E_8 и политоп Гоше 4_{21} : пакет из шестидесяти масок

10.1. Два H_4 -слоя

Политоп Гоше 4_{21} имеет 240 вершин, соответствующих 240 корням E_8 .

В статье [The Geometry of \$H_4\$ Polytopes](#) описывается представление этих корней посредством двух 600-ячейников:

$$H \quad \text{и} \quad \varphi H,$$

где:

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

— золотое сечение.

После редуцирования нормы две 120-элементные системы дают:

$$120 + 120 = 240$$

корней E_8 .

Эта связь также рассматривается в работах [From the Icosahedron to \$E_8\$](#) и [The Birth of \$E_8\$ out of the Spinors of the Icosahedron](#).

10.2. Иерархическая координата

Добавим номер золотого слоя:

$$\varepsilon \in \{0, 1\}.$$

Полная координата:

$$(\varepsilon, i, j, a),$$

где:

$$\begin{aligned} \varepsilon &\in \{0, 1\}, \\ i &\in \{0, \dots, 4\}, \\ j &\in \{0, \dots, 5\}, \\ a &\in \mathbb{Z}/4\mathbb{Z}. \end{aligned}$$

Число состояний:

$$2 \cdot 5 \cdot 6 \cdot 4 = 240.$$

Номер маски:

$$n = 30\varepsilon + 6i + j, \quad 0 \leq n < 60.$$

Арифметический декодер не меняется:

$$p_n = 10(q_0 + n) + r,$$

$$x_{n,a} = m_a p_n,$$

$$(k_{n,a}, d_{n,a}) = \text{divmod}(x_{n,a}, 10).$$

10.3. Результат

Один E_8 -макроблок содержит:

$$60 \text{ масок} \times 4 \text{ канала} = 240 \text{ состояний.}$$

Пакетное сокращение относительно независимых seed масок:

$$60\times.$$

Дополнительное укрупнение относительно одного 600-ячейника:

$$2\times.$$

Дополнительное укрупнение относительно одного 24-ячейника:

$$10\times.$$

Относительно независимых канальных seed:

$$240\times.$$

Последнее число нельзя смешивать с $60\times$:

- $60\times$ сравнивает один seed на маску с одним seed на макроблок;
- $240\times$ сравнивает четыре независимых канальных seed каждой маски с одним seed на макроблок.

11. Роль золотого сечения, дуальности и расслоения Хопфа

11.1. Золотое сечение

Золотое масштабирование создаёт второй H_4 -слой:

$$H \longleftrightarrow \varphi H.$$

Именно это даёт переход:

$$120 \longrightarrow 240$$

состояний и дополнительный пакетный множитель:

11.2. Что здесь называется дуалом

Нужно различать:

1. второй золотомасштабированный H_4 -слой в конструкции E_8 ;
2. геометрически двойственный 600-ячейнику 120-ячейник, имеющий 600 вершин;
3. антиподальную замену $v \mapsto -v$;
4. дуальность коротких и длинных корней.

Для 240 корней E_8 используется разложение на две 120-элементные H_4 -системы. Это не означает сложение 120 вершин 600-ячейника с 600 вершинами двойственного 120-ячейника.

11.3. Расслоение Хопфа

Расслоение Хопфа и кватернионная модель полезны как механизм:

- вычисления групповых состояний;
- перехода от единичных кватернионов к вращениям;
- организации слоёв и общих инвариантов;
- построения декодера без таблицы из 240 независимых записей.

Однако само слово «расслоение» не создаёт ещё один коэффициент сжатия.

Чтобы получить дополнительный множитель, волокна должны быть орбитами симметрии масок, а все их элементы должны однозначно вычисляться из одного представителя.

В текущей схеме антиподальная инволюция уже связана с зеркалом:

$$R = T^2.$$

Поэтому фактор $\{\pm 1\}$ нельзя второй раз считать как новое независимое $2\times$.

Вывод:

Расслоение Хопфа может сделать декодер красивее и дешевле, но подтверждённая ёмкость E_8 -макроблока остаётся равной 60 маскам, а не 120.

12. Точные коэффициенты оптимизации

12.1. Структурные коэффициенты

Переход	Коэффициент	Статус
Четыре независимых канала → C_4 -орбита	$4\times$	доказано
Отдельное зеркало → T^2	хранения не требуется	доказано
Два seed → B_4 -пакет	$2\times$	проверено
Шесть seed → 24-ячейник	$6\times$	проверено
Пять 24-ячейников → 600-ячейник	$5\times$ сверх 24-ячейника	точная групповая факторизация
Тридцать seed → 600-ячейник	$30\times$	точный аффинный пакет
Два H_4 -слоя → E_8	$2\times$ сверх 600-ячейника	точное разложение корней
Шестьдесят seed → E_8 -макроблок	$60\times$	точный аффинный пакет
24-ячейник → E_8 -макроблок	$10\times$ по ширине пакета	240/24, не автоматические $10\times$ по байтам

12.2. Почему общий результат не равен $4 \cdot 2 \cdot 6 \cdot 30 \cdot 60$

Эти числа относятся к разным базовым представлениям и вложенным уровням.

Правильная цепочка:

$4 \longrightarrow 24 \longrightarrow 120 \longrightarrow 240$

состояний, или:

$1 \longrightarrow 6 \longrightarrow 30 \longrightarrow 60$

масок.

Поэтому:

$$60 = 6 \cdot 5 \cdot 2,$$

а не произведение всех коэффициентов таблицы.

12.3. Сравнение с плотными битовыми половинами

Для N периодов:

$$p_n = 10(q_0 + n) + r$$

объём зеркальных половин равен:

$$B_{\text{half}} = 2 \sum_{n=0}^{N-1} (p_n - 1).$$

Если общий seed макроблока занимает S_N бит, то:

$$C_{\text{half} \rightarrow \text{seed}} = \frac{2 \sum_{n=0}^{N-1} (p_n - 1)}{S_N}.$$

Для E_8 -блока:

$$N = 60.$$

Точный численный коэффициент относительно плотных строк зависит от формата seed.

Поэтому корректное универсальное утверждение — $60\times$ по числу независимых seed масок, а не фиксированный коэффициент по битам для всех значений p .

12.4. Аффинный автомат не ограничен числом 60

Формула:

$$k_{q+n,r,a} = k_{q,r,a} + nm_a$$

работает для любого n .

Поэтому E_8 не является информационно-теоретическим пределом. Это удобный конечный макроблок:

- на 60 масок;
- на 240 канальных состояний;
- с развитой группой симметрий;
- с естественной иерархией $4 \rightarrow 24 \rightarrow 120 \rightarrow 240$.

Если требуется обработать M последовательных масок, один глобальный аффинный seed и длина диапазона могут породить все M масок. Тогда пакетный коэффициент относительно M независимых seed равен:

$$M \times.$$

Многогранники определяют удобную ширину вычислительного блока, но не предел арифметической рекурсии.

13. Что именно хранится

13.1. Глобальные константы формата

Один раз для всей программы задаются:

- множители:

$$(1, 3, 9, 7);$$

- канальный генератор C_4 ;
- правило зеркала $R = T^2$;
- трансверсаль C_4 в $2T$;
- трансверсаль $2T$ в $2I$;
- преобразование между слоями H и φH ;
- выбранное разбиение 600-ячейника на пять 24-ячейников.

Это код алгоритма или неизменяемая таблица, а не данные каждой маски.

13.2. Seed макроблока

Для блока последовательных периодов достаточно:

$$I = (q_0, r, N, \text{тип маски}).$$

Здесь:

- q_0 — начало блока;
- r — десятичный класс;
- N — число масок;
- тип определяет, строится G или применяется очистка P_E .

Для фиксированного E_8 -блока:

может быть частью формата и отдельно не храниться.

13.3. Prime-only режим

Аффинная орбита включает все:

$$p = 10q + r,$$

а не только простые p .

Если нужны исключительно простые периоды, требуется:

- селектор простых слоёв;
- либо применение составных периодов с последующей фильтрацией;
- либо внешний генератор допустимых периодов.

Селектор влияет на реальный битовый коэффициент, но не меняет правильность групповой адресации.

14. Алгоритмы и проверочные программы

14.1. Арифметический аффинный автомат

Файл:

```
examples_py/07_verify_affine_polytope_mapping.py
```

Проверяет:

- переход $p \mapsto p + 10$;
- сдвиги $(1, 3, 9, 7)$;
- восстановление слоёв $C_2 \times C_3$;
- 24 состояния шестимасочного блока;
- иерархию $5 \cdot 6 \cdot 4 = 120$ для 600-ячейника;
- иерархию $2 \cdot 5 \cdot 6 \cdot 4 = 240$ для E_8 .

Проверено:

39996

допустимых периодов до 100 000 и:

96000

состояний шестимасочных макроблоков, а также:

120000

состояний 600-ячейниковых блоков и:

240000

состояний E_8 -блоков.

14.2. Точная групповая проверка B_4/F_4

Файл:

```
examples_py/08_verify_b4_f4_group_mapping.py
```

В точной рациональной арифметике проверяет:

- подгруппу $C_4 \times C_2$ в B_4 ;
- порядок:

$$|W(F_4)| = 1152;$$

- порядок элемента Кокстера:

$$|c| = 12;$$

- две 12-элементные орбиты на корнях;
- транзитивную скрученную группу порядка 24;
- точные дуальности коротких и длинных корней.

14.3. Алгоритм 600-ячейникового блока

```
input:
    q0
    r

for i in 0 .. 4:          # пять 24-ячеек
    for j in 0 .. 5:      # шесть масок в 24-ячейнике
        n = 6*i + j
        p = 10*(q0+n) + r
```

```

for a in 0 .. 3: # четыре канала
    x = multiplier[a] * p
    emit(group_state=(i,j,a), value=x)

```

14.4. Алгоритм E_8 -блока

```

input:
    q0
    r

for epsilon in 0 .. 1: # Н и фН
    for i in 0 .. 4: # пять 24-ячеек
        for j in 0 .. 5: # шесть масок
            n = 30*epsilon + 6*i + j
            p = 10*(q0+n) + r

            for a in 0 .. 3:
                x = multiplier[a] * p
                emit(
                    root_state=(epsilon,i,j,a),
                    value=x
                )

```

Геометрическая координата вычисляется из фиксированных представителей классов:

$$(\varepsilon, i, j, a) \longmapsto \Phi_\varepsilon(g_i t_j c^a).$$

15. Рекомендуемая архитектура

15.1. Математическое ядро

Использовать аффинный групповой автомат:

$$(q, r, a) \longmapsto (q + n, r, a)$$

с кокциклом:

$$k \longmapsto k + nm_a.$$

15.2. Локальный уровень

Использовать:

$$C_4 \subset 2T$$

для кодирования:

$$6 \text{ масок} \times 4 \text{ канала.}$$

15.3. Средний уровень

Использовать:

$$2T \subset 2I$$

и одно разбиение Шоуте для кодирования:

$$5 \cdot 6 = 30$$

масок.

15.4. Верхний уровень

Использовать два слоя:

$$H \cup \varphi H$$

для кодирования:

$$2 \cdot 30 = 60$$

масок и:

$$60 \cdot 4 = 240$$

канальных состояний.

15.5. Практическая рекомендация

Для CPU/SIMD/GPU удобно выбрать:

- 24 состояния как малый блок;
- 120 состояний как средний блок;
- 240 состояний как крупный E_8 -блок.

Но хранить нужно не вершины и не рёбра, а:

- seed;
- номер внешнего блока;
- при необходимости prime-only селектор.

Все геометрические состояния вычисляются.

15.6. Оптимальный формат исполнительного слоя: хранить как E_8 , исполнять как 24-ячейники

Наиболее сбалансированный формат по памяти и скорости — один 60-битный selector на десятичный класс:

```
let active_masks: u64;
```

Используются младшие 60 бит:

$$\text{bit}_n = 1 \iff p_n = 10(q_0 + n) + r$$

включён в вычисление.

Оставшиеся четыре старших бита можно:

- оставить нулевыми;
- зарезервировать под версию или режим блока;
- использовать для локальных флагов, если перед сканированием выполняется маскирование младших 60 бит.

Для всех четырёх десятичных классов удобен блок:

```
#[repr(C, align(32))]
pub struct E8MaskBlock {
    selectors: [u64; 4],
}
```

Его размер:

$$4 \cdot 8 = 32 \text{ байта.}$$

Он описывает:

$$4 \cdot 60 = 240$$

кандидатных периодов и порождает до:

$$240 \cdot 4 = 960$$

канальных событий.

Если блоки расположены последовательно, значение q_0 не хранится в каждом блоке:

$$q_0 = q_{\text{global}} + 60 \text{ block_index}.$$

Сравнение selector-форматов

Для 60 масок одного десятичного класса:

Формат	Память	Особенность
десять byte-aligned 24-cell selector по 6 бит	10 байт	простой, но десять загрузок
плотно упакованные 24-cell selector	7,5 байта в среднем	минимум бит, неудобные границы и извлечение
один E_8 -selector	8 байт	одна выровненная загрузка и четыре свободных бита

Для четырёх десятичных классов:

Формат	Память на 240 периодов
byte-aligned 24-cell	40 байт
полностью bit-packed	30 байт
E_8 , четыре <code>u64</code>	32 байта

Следовательно, E_8 -формат:

- экономит 20% памяти относительно простого byte-aligned 24-cell формата;
- уступает предельно плотному битовому потоку только 2 байта на 240 периодов, то есть 6,25%;
- не требует невыровненных извлечений;
- целиком помещается в один 256-битный SIMD-регистр.

Горячий цикл

Для разреженного selector используется сканирование установленных битов:

```
let mut word = active_masks & ACTIVE_BITS;

while word != 0 {
    let n = word.trailing_zeros() as u64;
    word &= word - 1;

    let p = base_p + 10 * n;
    emit([p, 3 * p, 9 * p, 7 * p]);
}
```

Для полного или почти полного блока выгоднее последовательная рекурсия:

$$p_{n+1} = p_n + 10$$

и:

$$k_{n+1,a} = k_{n,a} + m_a.$$

Никакие матрицы E_8 , кватернионы или координаты вершин в горячем цикле не вычисляются.

Зачем сохранять 24-ячейниковое деление

60 бит логически делятся на десять подблоков:

$$60 = 10 \cdot 6.$$

Подблок:

$$\text{chunk}_j = (\text{word} \gg 6j) \& 63$$

является selector одного 24-ячейника.

Поэтому один и тот же формат поддерживает:

- крупное последовательное сканирование всего E_8 -блока;
- обработку десяти локальных 24-ячейников;
- таблицу из 64 вариантов локального 6-битного состояния;
- параллельную выдачу подблоков разным вычислительным потокам.

Практический выбор:

В RAM хранить один `u64` на 60 масок каждого десятичного класса. Рассматривать его как E_8 -блок, но при необходимости делить на десять вычисляемых 24-ячейниковых подблоков.

Это гибрид:

E_8 для хранения + 24-ячейник для локального исполнения.

Rust-реализация и воспроизводимый benchmark находятся в:

`rust/e8_mask_codec/`.

На CPU проверены три режима:

1. `Sparse` — `trailing_zeros` и сброс младшего установленного бита;
2. `Dense` — последовательный проход по 60 позициям;
3. `Cell24Lookup` — таблица 64 вариантов для 6-битного подблока.

На текущем процессоре:

- `sparse`-путь выигрывает примерно до 50% заполнения;
- `dense`-путь имеет смысл только для очень плотных сегментов;
- 24-cell LUT проигрывает прямому битовому сканированию на CPU.

Режим выбирается один раз для целого сегмента, а не отдельно для каждого `u64`. Это устраняет диспетчеризацию из горячего цикла.

Промежуточный `Vec` событий не создаётся: callback должен сразу изменять целевой битсет решета.

15.7. Какие предложения из `optimizations.md` использованы

Файл `docs/optimizations.md` является внешним пересказом этой же схемы, а не источником исходной конструкции. До данного сравнения он не использовался.

После проверки на Rust:

Предложение	Решение
один 60-битный selector	используется

Предложение	Решение
sparse scan через <code>trailing_zeros</code>	используется
отдельный dense-путь	используется для сегментов плотнее примерно 50%
LUT из 64 паттернов 24-cell	отвергнута для CPU hot path; отдельно проверяется на GPU
прямое наложение на битсет	используется через callback без промежуточного массива
SIMD	следующий уровень после появления конкретного layout целевого решета

Утверждение, что `ctz` всегда выполняется за один такт, не является переносимой гарантией. Также `popcnt` считает биты, но сам по себе не устанавливает позиции в целевом решете.

15.8. Параллельная архитектура CPU + GPU

CUDA-часть реализована по архитектурному шаблону проекта:

`/media/ilja/DATA/soft/fastsearch`.

Rust отвечает за:

- формат данных;
- разбиение диапазона;
- выбор CPU/GPU-режима;
- проверку результатов;
- benchmark и политику памяти.

CUDA-ядро собирается `nvcc` через `build.rs`. В пользовательском API и документации используются Rust-типы.

На GTX 1660 Ti проверены:

1. `Sparse` — один поток CUDA на 60-битное слово;
2. `DenseWarp` — один warp на слово;
3. `Cell24Lookup` — constant-memory LUT для 6-битных чанков.

Результаты decoder benchmark:

Плотность	Лучший GPU-режим
5–95%	<code>Sparse</code>
около 99–100%	<code>Cell24Lookup</code>
все измеренные плотности	<code>DenseWarp</code> не победил

Но полностью заполненный блок следует кодировать как:

`ImplicitFull`,

то есть вообще без `selector`. Поэтому LUT остаётся специализированным экспериментальным режимом, а основной GPU-путь — `Sparse`.

Конвейер

CPU и GPU не должны обрабатывать одни и те же блоки. Диапазон делится на непересекающиеся сегменты:

$$\mathcal{B} = \mathcal{B}_{\text{CPU}} \sqcup \mathcal{B}_{\text{GPU}}.$$

Одновременно:

1. GPU обрабатывает крупный текущий пакет;
2. CPU обрабатывает свою долю и хвосты;
3. CPU готовит следующий закреплённым буфером;
4. следующий CUDA stream принимает пакет;
5. ядро сразу изменяет device-bitset без выгрузки списка событий.

CUDA context, streams и буферы должны быть постоянными. Холодная инициализация не входит в steady-state алгоритм.

После перевода гибридного пути на постоянный пул CUDA и device-resident режим без обратной выгрузки массива событий steady-state benchmark на 4 194 304 selector-словах при плотности 25% и разбиении 30% CPU / 70% GPU дал ускорение порядка:

$$2,8\text{--}3,2\times$$

относительно только CPU на 8 потоках. Это уже не allocating-прототип: `cudaMalloc` и `cudaFree` убраны из горячего пути, pinned-буферы и device-буферы переиспользуются.

15.9. Память CPU + GPU

Для одного E_8 -блока из 240 периодов:

Формат	Байт	Бит на период
ImplicitFull	0	0
ideal bit-packed	30	1
E8Bitmap	32	1,0667
byte-aligned 24-cell	40	1,3333

Для очень разреженных данных используется CSR:

$$B_{CSR} = 4(B + 1) + A,$$

где:

- B — число E_8 -блоков;
- A — число активных периодов;
- один индекс занимает один байт, потому что $0 \leq n < 240$.

CSR выигрывает у E8Bitmap приблизительно при плотности ниже:

$$\frac{28}{240} \approx 11,7\%.$$

Итоговый выбор:

Плотность/тип	Хранение
100%	ImplicitFull
менее примерно 11,7%	SparseCSR
остальное	E8Bitmap

Для совместной работы CPU+GPU полный набор не нужно дублировать в RAM и VRAM. При постоянном разбиении:

$$B_{\text{host}} + B_{\text{device}} = B_{\text{all}}.$$

Дополнительная память требуется только для небольшого двойного закреплённым буфером.

Для миллиона E_8 -блоков:

- `E8Bitmap` : 30,518 MiB;
- ideal bit-packed: 28,610 MiB;
- byte-aligned 24-cell: 38,147 MiB;
- `SparseCSR` при 10%: 26,703 MiB;
- `ImplicitFull` : 0 байт selector.

Rust-калькулятор:

```
rust/e8_mask_codec/src/bin/memory_report.rs.
```

Итоговая подтверждённая иерархия:

$$C_4 \subset 2T \subset 2I \subset E_8$$

и:

$$1 \longrightarrow 6 \longrightarrow 30 \longrightarrow 60 \text{ масок на seed.}$$

16. Текущее состояние реализации и измеренные тесты производительности

Текущая рабочая инженерная база живёт в crate:

```
rust/e8_mask_codec/.
```

Сейчас в репозитории уже есть не только математическая схема, но и измеряемые компоненты исполнительного слоя:

- `bench_cpu_threads.rs` — базовый CPU-замер на 1/8/12 потоках;
- `bench_gpu.rs` — бенчмарк GPU-декодера для `Sparse`, `DenseWarp` и `Cell24Lookup`;
- `bench_hybrid.rs` — совместная работа CPU+GPU после внедрения постоянного пула CUDA;
- `memory_report.rs` — выбор между `ImplicitFull`, `SparseCSR` и `E8Bitmap`;
- `count_first_n.rs` — новый исполнительный слой подсчёта с контрольными точками для первых n простых.

16.1. CPU

На текущей машине `bench_cpu_threads` показывает порядок величин:

- 1 поток: около 38.8 Mword/s;
- 8 потоков: около 205–211 Mword/s;
- 12 потоков: режим нестабилен и зависит от SMT/планировщика.

Поэтому производственным ориентиром для текущего железа следует считать именно 8 потоков, а не 12.

16.2. GPU

На GTX 1660 Ti основной путь декодера — `Sparse`.

По текущему `bench_gpu`:

- при плотности 5% `Sparse` даёт около 10.1 Gword/s;
- при плотности 25% — около 3.55 Gword/s;
- при плотности 50% — около 2.40 Gword/s.

`Densewarp` не победил ни на одной из измеренных плотностей. `Cell24Lookup` становится выгодным только в почти полном режиме около 99–100%, но такие блоки в рабочем режиме логичнее кодировать как `ImplicitFull`.

16.3. Устойчивый гибридный режим

После внедрения постоянного пула CUDA и режима с резидентными данными на GPU `bench_hybrid` показывает ускорение порядка 2.8–3.2× относительно режима только CPU на 8 потоках. Именно этот диапазон следует считать актуальным измеренным ориентиром, а не старые цифры прототипа с постоянными выделениями памяти.

16.4. Память

`memory_report` подтверждает текущую политику выбора хранения:

- ниже примерно 11.7% плотности выигрывает `SparseCSR`;
- в средней зоне лучше `E8Bitmap`;
- при 100% плотности лучшим представлением становится `ImplicitFull`.

Для 1 000 000 E_8 -блоков:

- `E8Bitmap`: 30.518 MiB;
- `SparseCSR` при 10%: 26.703 MiB;

- `ImplicitFull` при 100%: 0 байт selector;
- разбиение CPU+GPU с двойным закреплённым буфером: 31.518 MiB суммарно.

17. Практический рубеж: первые 10^{12} простых

Под фразой «вычислить триллион простых» здесь понимаются именно **первые** 10^{12} простых чисел, а не все простые, не превосходящие 10^{12} .

Эти две задачи радикально различаются:

- все простые $\leq 10^{12}$ — это сравнительно лёгкий диапазонный подсчёт;
- первые 10^{12} простых доходят до числа порядка $3 \cdot 10^{13}$.

Контрольное целевое значение:

$$p_{10^{12}} = 29\,996\,224\,275\,833.$$

Это хороший рубеж по трём причинам.

17.1. Он ещё реалистичен на одной машине

Для этой задачи верхняя оценка лежит в области $\sim 3 \cdot 10^{13}$, поэтому достаточно базы делителей лишь до:

$$\sqrt{p_{10^{12}}} \approx 5.48 \cdot 10^6.$$

Это на много порядков легче, чем сценарии вроде полного прохода до 10^{20} или тем более до 10^{30} .

17.2. Он проверяет всю инфраструктуру, а не только одну формулу

Рубеж «первые 10^{12} простых» проверяет одновременно:

- сегментацию;
- выбор формата хранения;
- контрольные точки и продолжение;
- CPU-путь исполнительного слоя;
- дальнейший переход к исполнительному слою CPU+GPU.

17.3. Полная выгрузка не нужна как первый режим

По умолчанию вывод должен идти в потоковом режиме с контрольными точками:

- `found_count` ;
- `last_prime` ;
- `processed_segments` ;
- статистика хранения;
- файлы контрольных точек.

Именно так сейчас устроен новый runner:

```
rust/e8_mask_codec/src/bin/count_first_n.rs .
```

Текущие проверочные примеры уже подтверждены:

- $p_{1000} = 7919$;
- $p_{100000} = 1299709$;
- $p_{10^6} = 15485863$.

17.4. Сохраняемое состояние, продолжение и зафиксированный сегмент

Текущий исполнительный слой уже умеет считать длинную префиксную задачу не «одним заходом», а в режиме сохраняемого состояния. Для этого поддерживаются:

- файл контрольной точки с атомарной записью через временный файл и `rename` ;
- файл прогресса в человекочитаемом виде;
- `--resume` и `--no-resume` ;
- сохранение по числу сегментов и по таймеру одновременно;
- корректная остановка по `SIGINT` и `SIGTERM` .

В контрольной точке сейчас сохраняются именно те поля, которые позволяют продолжить прогон не с нуля, а с последней зафиксированной границы:

- `target_count` ;
- `found_count` ;
- `last_prime` ;
- `next_q` ;
- `processed_segments` ;
- `segment_blocks` ;
- `backend` как имя параметра режима исполнения;
- `thread_count` ;
- `checkpoint_every_segments` ;

- `checkpoint_interval_sec`;
- накопленное `elapsed_millis`;
- счётчики хранилища по `ImplicitFull`, `SparseCSR`, `E8Bitmap`.

Единицей надёжной фиксации сейчас является **завершённый сегмент**. Это и есть практический смысл зафиксированного сегмента в текущем CPU-исполнительном слое: сначала сегмент полностью досчитывается, затем его вклад мержится в глобальные счётчики, и только после этого контрольная точка может считаться зафиксированной.

Если процесс завершается между двумя контрольными точками, теряется только последнее незакоммиченное окно, а не весь прогон.

17.5. Практические команды для фонового прогона

Типовой запуск нового длинного прогона:

```
cargo run --release --bin count_first_n -- \
  --n 1000000000000 \
  --backend cpu \
  --threads 8 \
  --segment-size 4096 \
  --checkpoint data/count_1e11.checkpoint \
  --progress data/count_1e11.progress \
  --checkpoint-every 64 \
  --checkpoint-interval-sec 60 \
  --progress-interval-sec 15
```

Запуск в фоне через `nohup`:

```
nohup cargo run --release --bin count_first_n -- \
  --n 1000000000000 \
  --backend cpu \
  --threads 8 \
  --checkpoint data/count_1e11.checkpoint \
  --progress data/count_1e11.progress \
  --resume \
  > data/count_1e11.log 2>&1 &
```

Продолжение после остановки:

```
cargo run --release --bin count_first_n -- \
  --n 1000000000000 \
  --backend cpu \
  --threads 8 \
  --checkpoint data/count_1e11.checkpoint \
  --progress data/count_1e11.progress \
  --resume
```

Смотреть текущее состояние можно без разбора бинарного формата:

```
cat data/count_1e11.progress
```

На текущем этапе исполнительный слой уже умеет исполняться во всех трёх режимах `backend=cpu`, `backend=gpu` и `backend=hybrid`, то есть эти ветки больше не являются фиктивными флагами. Поток надёжного архива `--archive-dir` использует модуль записи chunk-файлов с delta-кодированием и учётом контрольных точек, `manifest.txt`, `index.tsv` и безопасное продолжение после остановки без пересоздания уже зафиксированных блоков на диске.

18. Где оптимизации реально дают выигрыш

18.1. Один канал вместо четырёх

Это первое точное сокращение. Оно уменьшает материализованное состояние в четыре раза и одновременно упрощает адресацию. Остальные три канала восстанавливаются циклическим действием, а не хранятся независимо.

18.2. Зеркало без отдельного хранения

Зеркало уже сидит внутри канальной симметрии и поэтому не даёт отдельного множителя $2\times$ сверху. Выигрыш возникает не как «ещё одно независимое сжатие», а как отказ от дублирования одной и той же структуры.

18.3. `E8Bitmap`, `SparseCSR`, `ImplicitFull`

На практике это сейчас главный переключатель формата памяти:

- `SparseCSR` — для реально разреженных сегментов;
- `E8Bitmap` — базовый плотный формат;
- `ImplicitFull` — для полностью заполненных блоков.

Именно здесь возникают уже измеренные выигрыши по памяти, а не только геометрическая интерпретация.

18.4. CPU, GPU и гибридный режим

- режим только CPU хорош как базовый ориентир и для небольших/нерегулярных задач;
- только GPU хорош для крупных однородных пакетов;
- гибридный режим выигрывает, когда диапазон разбит на непересекающиеся части и GPU не тратит время на обратную выгрузку списка событий.

18.5. Почему LUT не нужен в горячем пути CPU

На CPU `Cell24Lookup` проигрывает `trailing_zeros + selector &= selector - 1`. Поэтому LUT следует считать исследовательским режимом, а не основным рантаймом.

18.6. Почему постоянный пул CUDA обязателен

Как показали замеры, переход от прототипа с постоянными выделениями памяти к persistent pool меняет картину качественно. Без него бенчмарк измеряет не алгоритм, а накладные расходы `cudaMalloc` / `cudaFree`.

19. Почему диапазон $10^{19} \rightarrow 10^{20}$ не является первой практической целью

Математически этот диапазон, конечно, достижим. Но как первый инженерный рубеж он плох.

Между 10^{19} и 10^{20} находится:

$$10^{20} - 10^{19} = 9 \cdot 10^{19}$$

целых чисел. После исключения классов, кратных 2 и 5, остаётся:

$$0.4 \cdot 9 \cdot 10^{19} = 3.6 \cdot 10^{19}$$

кандидатов.

Даже если предположить суммарную скорость уровня:

- 10^{10} кандидатов/с,
- $3 \cdot 10^{10}$ кандидатов/с,
- 10^{11} кандидатов/с,

то порядок времени остаётся таким:

- около 114 лет;
- около 38 лет;
- около 11.4 года.

Поэтому диапазон $10^{19} \rightarrow 10^{20}$ полезен как **теоретическая шкала масштаба**, но не как первая задача, на которой стоит отлаживать исполнительный слой.

Практически намного полезнее:

1. реализовать первые 10^{12} простых;
2. затем локальный поиск отдельных больших простых;
3. и только потом думать о гигантских непрерывных диапазонах.

20. Большие отдельные простые: `isPrime(N)` и `nextPrime(N)`

Этот раздел описывает текущий пользовательский слой для проверки и поиска больших отдельных простых чисел. Он дополняет префиксный счётчик первых n простых и работает как локальный запрос около заданного числа N .

Главное разделение такое:

- быстрый режим даёт практический ответ через локальную маску и Miller–Rabin;
- уточнённый режим даёт доказательный ответ через ECPP-сертификат;
- архивный режим остаётся точным внутри уже покрытого `wheel210`-архива.

20.1. Что уже измерено

Уже измерены и подтверждены базовые строительные блоки:

- кодек состояний;
- режимы сканирования CPU;
- разреженный GPU-декодер;
- устойчивый гибридный режим CPU+GPU;
- политика выбора формата хранения;

- префиксный исполнительный слой с контрольными точками для первых n простых.

Эти компоненты ускоряют масочную и сегментную часть вычислений. Финальная проверка 1000-значного кандидата через Miller–Rabin или ECPP использует арифметику больших целых чисел.

20.2. Текущая поверхность запросов

В исполнительном слое есть два связанных, но разных слоя запросов.

Первый слой — архивные запросы:

```
make isPrime N=...  
make nextPrime N=...
```

Если число попадает в покрытый диапазон зафиксированного `wheel210` -архива, ответ берётся из архива и является точным для этого диапазона. Если запрос выходит за покрытый диапазон, исполнительный слой возвращает явную ошибку, а не скрывает отсутствие данных случайным обходным путём.

Второй слой — запросы к большим целым числам до 1024 десятичных знаков. Этот слой работает без прохода от нуля к N : для заданного числа строится локальное масочное состояние, затем выполняется быстрая или доказательная проверка.

20.3. Чем локальный запрос отличается от полного префиксного прохода

Для числа порядка 10^{1000} интересна не генерация всех предыдущих простых, а локальный вход в область около N :

- входное число N переводится в `wheel/channel/segment state`;
- для малой ситовой базы вычисляются локальные `offsets`;
- строится масочное поле только для нужного локального сегмента;
- выжившие кандидаты проверяются Miller–Rabin;
- по запросу уточнения финальный кандидат получает ECPP-сертификат.

Такой режим не строит историю масок и не проходит числовой ряд от нуля.

20.4. Ограничение версии для больших целых

Реализованный слой больших целых ограничен числами:

```
max_decimal_digits = 1024
```

Ограничение относится к:

- `isPrime(N)` ;
- `nextPrime(N)` ;
- уточнению результата через ECPP.

Для входов больше 1024 десятичных знаков возвращается явная ошибка о превышении поддерживаемого размера. В этом документе раньше обсуждались масштабы 10^{1000} и 10^{10000} как архитектурная граница локального входа в дальний диапазон. Текущий рабочий слой намеренно зафиксирован на 1024 десятичных знаках, чтобы время, поведение и интерфейс оставались управляемыми.

20.5. Маска как доказательство и маска как фильтр

Маска является доказательством простоты только в полном решетном смысле:

если для кандидата x учтены все простые делители $p \leq \sqrt{x}$,
и x выжило,
то x простое.

Для 1000-значного числа такая полная база практически недоступна:

$$\sqrt{10^{1000}} = 10^{500}.$$

Поэтому в слое запросов к большим целым локальная маска работает как быстрый детерминированный фильтр. Она отбрасывает кандидатов, делящихся на малые простые из выбранной базы, но сама по себе не превращает выжившего кандидата в доказанное простое число.

Итоговая логика:

Слой	Что означает выживание кандидата
полная база до $\sqrt{x}$	доказательство простоты через решето
малая база	кандидат не делится на малые простые

Слой	Что означает выживание кандидата
малая база + Miller–Rabin	практическое вероятно простое
малая база + Miller–Rabin + ECPP	доказанное простое

Ключевой принцип остаётся прежним:

Маски локальны по позиции, но не бесплатны по базе.

20.6. Быстрый `isPrime(N)`

Обычный пользовательский вызов `isPrime(N)` выполняет быстрый путь:

- wheel-210;
- маска по малым простым;
- Miller–Rabin, обычно 16–32 раунда;
- возврат результата.

Возможные статусы:

Статус	Смысл
Составное	найден малый делитель или свидетель составности Miller–Rabin; ответ точный
Вероятно простое	число прошло маску и Miller–Rabin; это практический вероятностный ответ

Если Miller–Rabin нашёл свидетель составности, ответ `Составное` является строгим. Если число прошло Miller–Rabin, быстрый ответ остаётся вероятностным, а не сертификатом простоты.

В интерфейсе рядом с быстрым ответом доступно действие `Уточнить ответ`. Оно запускает доказательную проверку через ECPP.

20.7. Уточнённый `isPrime(N)`

Уточнение использует быстрый ответ как предварительный фильтр:

- если быстрый режим уже нашёл составность, результат остаётся **Составное** ;
- если быстрый режим вернул **Вероятно простое** , запускается ЕСРР;
- сгенерированный ЕСРР-сертификат проверяется перед показом результата;
- успешное уточнение даёт статус **Доказано простое** .

Такой режим отделяет быстрый практический ответ от строгого математического подтверждения.

20.8. Быстрый **nextPrime(N)**

Обычный **nextPrime(N)** ищет первое практическое вероятно простое после N через локальные сегменты:

- старт нормализуется к ближайшему допустимому wheel-кандидату;
- вокруг старта создаётся локальный сегмент;
- wheel-210 и малые маски удаляют очевидные составные;
- выжившие кандидаты сканируются в порядке возрастания;
- Miller–Rabin применяется только к выжившим кандидатам;
- первый кандидат, прошедший Miller–Rabin, возвращается как быстро найденное вероятно простое число.

Если текущий сегмент не содержит подходящего кандидата, исполнительный слой расширяет поиск следующим локальным сегментом.

20.9. Уточнённый **nextPrime(N)**

Уточнение для **nextPrime(N)** применяется только к финальному кандидату, найденному быстрым режимом.

Рабочая схема:

1. локальная маска находит выживших кандидатов;
2. Miller–Rabin выбирает первый вероятно простой кандидат P ;
3. действие **Уточнить ответ** запускает ЕСРР только для этого P ;
4. после успешной проверки сертификата результат получает статус **Доказано простое** .

ЕСРР на каждый выживший кандидат не используется: это слишком дорогой путь и он не нужен для текущей пользовательской семантики.

20.10. Оценка времени для 1000-значных чисел

Оценка относится к следующей конфигурации:

- CPU: i7, 6 ядер / 12 потоков;
- RAM: 64 GB;
- GPU: Nvidia GeForce GTX 1660 Ti;
- механизм больших целых: Rust + GMP;
- размер N : около 1000 десятичных знаков;
- ограничение версии: не более 1024 десятичных знаков.

Задача	Метод	Примерное время на 1000 знаков
<code>isPrime(N)</code>	маска + Miller–Rabin 16–32	0.05 – 1 сек
<code>isPrime(N)</code>	маска + Miller–Rabin 64	0.2 – 3 сек
<code>isPrime(N)</code>	маска + ECPP	10 сек – 10 мин
<code>nextPrime(N)</code>	маска + Miller–Rabin	1 – 30 сек
<code>nextPrime(N)</code>	маска + MR + ECPP финального кандидата	20 сек – 10+ мин

Это инженерные оценки, а не SLA. Реальное время зависит от числа раундов Miller–Rabin, глубины маски малых простых, размера сегмента для `nextPrime`, числа выживших кандидатов и удачности генерации ECPP-сертификата.

20.11. Роль CPU/GPU оптимизаций

CPU/GPU оптимизации проекта ускоряют главным образом массовую масочную часть:

- разметку кратных;
- работу по wheel residue-классам;
- сканирование `E8Bitmap`, `SparseCSR` и `ImplicitFull`;
- обработку непересекающихся сегментов;
- гибридный режим для больших сегментных задач.

Для одного конкретного `isPrime(N)` на 1000 знаков GPU почти не ускоряет финальную проверку: основная стоимость находится в GMP `pow_mod` и в арифметике больших целых,

используемой ЕСРР.

Практическое разделение такое:

Операция	Где основной выигрыш
<code>isPrime(N)</code>	CPU + GMP; малая маска как быстрый отсев
<code>nextPrime(N)</code>	сегментная маска; CPU/GPU/гибрид при достаточно большом сегменте
ЕСРР	арифметика больших целых и доказательство на эллиптических кривых; E8/CUDA-декодер его не заменяет

20.12. Прогресс вычислений

В интерфейсе прогресс появляется, когда операция длится дольше 3 секунд.

Для Miller–Rabin прогресс точный:

```
completed_rounds / total_rounds
```

Для сегментной маски прогресс точный внутри текущего сегмента:

```
processed_blocks / total_blocks
```

Для `nextPrime(N)` общий процент до найденного простого заранее неизвестен, поэтому отображается стадийный прогресс текущего сегмента: построение маски, сканирование выживших кандидатов, Miller–Rabin для текущего кандидата и переход к следующему сегменту.

Для ЕСРР общий процент может быть только стадийным или оценочным. Вместо фиктивной точности интерфейс показывает текущую стадию, прошедшее время, число попыток и размер текущей подзадачи. Если процент оценочный, он помечается как оценочный.

20.13. ЕСРР не является ML/тригонометрическим вычислением

ЕСРР не использует sigmoid и не является нейросетевым вычислением.

Основная нагрузка находится здесь:

- арифметика больших целых;

- модульное умножение;
- модульное возведение в степень;
- НОД;
- арифметика эллиптических кривых по модулю N ;
- умножение точки на число;
- частичная факторизация порядков групп;
- генерация сертификата;
- проверка сертификата.

Некоторые реализации Atkin–Morain/CM могут использовать вспомогательные высокоточные комплексные вычисления при построении данных, но пользовательский исполнительный слой этого слоя в основном нагружает целочисленную и модульную арифметику.

20.14. Команды и статусы

Основные команды слоя запросов:

```
make isPrime N=...
make nextPrime N=...
```

В интерфейс эти команды соответствуют быстрым действиям. Рядом с быстрым результатом доступно уточнение через ECPP.

Типовые статусы:

Статус	Где появляется
Составное	<code>isPrime</code> , если найден делитель или свидетель составности
Вероятно простое	быстрый <code>isPrime</code> или быстрый <code>nextPrime</code> после Miller–Rabin
Доказано простое	после успешного ECPP-уточнения
Уточнение недоступно	если модуль ECPP не собран или не подключён

Если модуль ECPP недоступен, результат не подделывается: быстрый ответ остаётся вероятностным, а интерфейс сообщает, что доказательное уточнение недоступно.

20.15. Итоговая семантика слоя больших целых

Вызов	Быстрый режим	Уточнённый режим
<code>isPrime(N)</code>	Составное или Вероятно простое	Составное или Доказано простое
<code>nextPrime(N)</code>	первый вероятно простой кандидат после N	тот же кандидат после ECPP-сертификата

Ключевая инженерная позиция:

Не считать предыдущие маски.

Не идти от 0 до N .

Использовать локальные offsets и сегментные маски.

Быстрый ответ давать через Miller-Rabin.

Доказательный ответ давать через ECPP только по запросу уточнения.

21. Практические сценарии применения

21.1. Последовательный префиксный поиск

Цель: получить первые n простых.

Режим:

- `count_first_n --n ...;`
- потоковое выполнение с контрольными точками;
- при необходимости `--archive-dir ...` для chunk-архива с delta-кодированием на диске;
- контроль по `found_count`, `last_prime`, `archive_committed_primes`.

Это лучший первый производственный рубеж.

21.2. Проверка конкретного большого числа

Цель: проверить `isPrime(N)` для одного большого N .

Режим:

- не полный проход от нуля;
- локальная инициализация масочного состояния;

- быстрый ответ через маску и Miller–Rabin;
- доказательное уточнение через ECPP.

21.3. Поиск первого простого после N

Цель: `nextPrime(N)`.

Режим:

- локальные сегменты вокруг N ;
- повторная инициализация фаз;
- возврат первого выжившего кандидата, прошедшего Miller–Rabin;
- ECPP-сертификат только по запросу уточнения.

21.4. Гибридный путь

Гибридный режим полезен, когда:

- задача велика;
- сегменты можно развести между CPU и GPU без пересечения;
- результат не нужно возвращать как длинный список событий через PCIe.

21.5. Кластер и удалённые диапазоны

Кластерный режим имеет смысл для очень длинных префиксных задач или для большого пула независимых запросов `nextPrime(N)`. Но как первая практическая цель он сложнее, чем исполнительный слой на одной машине на одной машине.

22. Источники

22.1. Внутренние ссылки на текущие расчёты

1. `rust/e8_mask_codec/src/bin/bench_cpu_threads.rs` — базовый замер по потокам CPU.
2. `rust/e8_mask_codec/src/bin/bench_gpu.rs` — GPU sparse/dense/LUT.
3. `rust/e8_mask_codec/src/bin/bench_hybrid.rs` — устойчивый режим CPU+GPU с постоянным pool.
4. `rust/e8_mask_codec/src/bin/memory_report.rs` — политика памяти.
5. `rust/e8_mask_codec/src/bin/count_first_n.rs` — исполнительный слой с контрольными точками для первых n простых.

6. `rust/e8_mask_codec/src/runtime.rs` — сегментированный исполнительный слой подсчёта и состояние контрольной точки.

22.2. Внешние математические источники

1. T. Denney et al., [The Geometry of \$H_4\$ Polytopes](#). Разбиения 600-ячейника на пять непересекающихся 24-ячейников и связь с E_8 .
2. J. C. Baez, [From the Icosahedron to \$E_8\$](#) . Икосианы, золотое сечение, 600-ячейник и решётка E_8 .
3. P.-P. Dechant, [The Birth of \$E_8\$ out of the Spinors of the Icosahedron](#). Спинорная конструкция H_4 и E_8 .
4. M. Koca, M. Al-Ajmi, N. O. Koca, [Quaternionic Representation of Snub 24-Cell and its Dual Polytope Derived From \$E_8\$ Root System](#). Кватернионные представления 24-ячейника, 600-ячейника и их орбит.

23. Практический запуск длинного прогона через Makefile

Для длинного прогона используется корневой `Makefile`.

Основные команды:

```
make prime-build
make count-start
make count-status
make count-stop
make count-log
make count-wheel210-rebuild
```

`make count-start` — универсальная команда старта и продолжения. Она работает даже без заранее созданного файла контрольной точки:

- если существует надёжный архив, состояние берётся из архивной истины;
- иначе, если есть контрольная точка, состояние продолжается из неё;
- иначе создаётся свежее начальное состояние.

`make count-status` показывает:

- утверждение текущего прогона из `count.progress`;

- зафиксированное состояние архива;
- буферизированное состояние в RAM;
- ETA и скорость, рассчитанные по модели «архив + буфер»;
- текущий размер `wheel210` на диске.

Таким образом, длинный прогон управляется как нормальный фоновый вычислительный процесс, а не как одноразовый интерактивный запуск.

23.1. Wheel210-only режим хранения

Текущий производственный режим архива использует `--archive-mode wheel210`, чтобы не держать на диске одновременно точный архив с delta-кодированием и побочный `wheel210`-архив.

Именно `wheel210-only` соответствует целевой оценке порядка `20–40 GiB` на `1011` простых. Режим `both` сохраняется только как переходный или отладочный.

24. Приложение: текущий исполнительный и операционный слой

Это приложение фиксирует текущий исполнительный слой и слой эксплуатации, не меняя математическую часть документа.

В реализации сейчас есть:

- универсальный интерфейс Makefile для длинного префиксного поиска;
- архивные запросы к простым числам;
- честная трёхслойная модель статуса;
- локальное состояние сборки и кэша проекта внутри `data/`;
- `wheel210-only` как основной производственный режим архивного хранения.

25. Универсальные операции Makefile

Публичный операционный интерфейс — параметризованный корневой `Makefile`.

Основные команды:

```
make prime-build
make test
make count-start
```

Вытросс вычисление простых чисел
make count-stop
Групповое кодирование циклических масок, локальные запросы и архивный runtime
make count-status

```
make count-progress  
make count-log  
make isPrime N=2760727302517  
make nextPrime N=2760727302510
```

Те же команды используются для альтернативных прогонов через переопределение переменных:

- `RUN_DIR` ;
- `TARGET_N` ;
- `ARCHIVE_DIR` ;
- `CHECKPOINT_FILE` ;
- `PROGRESS_FILE` ;
- `PID_FILE` ;
- `LOG_FILE` .

Отдельные специализированные псевдокоманды для `start` / `stop` / `status` в рабочем режиме не требуются.

26. Модель истины для длинных прогонов

`make count-status` разделяет три слоя истины. Слой запросов также доступен через локальный HTTP-интерфейс.

26.1. Утверждение текущего прогона

Это то, что сообщает текущий файл `count.progress` :

- запрошенная цель;
- последнее заявленное значение `found_count` ;
- последнее заявленное значение `last_prime` ;
- текущий режим исполнения и телеметрия исполнительного слоя.

26.2. Зафиксированная истина архива

Это авторитетное зафиксированное состояние на диске:

- итоговые значения из manifest `wheel210` ;
- число зафиксированных chunk-файлов;
- диапазон архива, доступный для запросов;
- текущий размер архива на диске.

26.3. Буферизированная истина в RAM

Это работа, которая уже найдена, но ещё не зафиксирована надёжной записью:

- простые, находящиеся в RAM-буфере;
- оценочный размер RAM-буфера;
- состояние очереди записи/сжатия.

Такое разделение не смешивает устаревшие утверждения процесса с зафиксированным состоянием архива.

27. Архивные запросы к простым числам

Проект предоставляет архивный поиск простых через существующие CLI-команды:

```
make isPrime N=...
make nextPrime N=...
```

Поведение:

- если `N` покрыто `wheel210` -архивом, ответ точный;
- `nextPrime(N)` возвращает первое архивное простое строго больше `N` ;
- пропуски архива или запросы за пределами диапазона возвращают явные ошибки;
- пользовательский вывод чистый и ориентирован на результат.

Это уже не будущая цель API: архивный слой запросов существует и входит в текущую поверхность исполнительного слоя.

28. Текущая рабочая форма исполнительного слоя

Текущий исполнительный слой использует следующие константы и политики:

- базовый CPU-режим: 8 потоков;

- целевой устойчивый GPU-режим: примерно 131072 активных потоков;
- рабочие потоки архивной записи: 3;
- RAM-буферизированный архивный конвейер с обратным давлением;
- `wheel210-only` как производственный режим архива;
- локальная среда сборки и кэша проекта внутри `data/`.

Операционно это означает:

- новое состояние проекта не пишется в `$HOME` ;
- кэш сборки живёт в `data/cargo-home` и `data/cargo-target` ;
- вспомогательные файлы исполнительного слоя живут внутри выбранного `RUN_DIR` ;
- продолжение из архива предпочтительно, если существует зафиксированное состояние архива.

29. Цели и нейтральная дорожная карта

Активные цели исполнительного слоя и хранилища:

- сжатый архив меньше `40 GiB` ;
- гибридная производительность выше только CPU на прогонах с архивом, приближённых к рабочему режиму;
- продолжение использования `wheel210-only` для надёжный архива;
- будущая полная переупаковка/пересжатие архива как следующий крупный шаг по хранилищу.

Эти цели зафиксированы как активные инженерные ориентиры без превращения приложения в отчёт о регрессиях.

30. Локальный интерфейс для `isPrime(N)` и `nextPrime(N)`

Для архивных запросов к простым числам доступен локальный одностраничный интерфейс. Первая версия намеренно локальная и привязана к `127.0.0.1`.

30.1. Что показывает интерфейс

Интерфейс открывает две операции:

- `isPrime(N)` ;

- `nextPrime(N)` .

Обе операции вызывают исполнительный слой Rust напрямую. Если запрошенное число покрыто зафиксированным `wheel210` -архивом, ответ возвращается из архивного пути запросов. Если архив не настроен, исполнительный слой может использовать прямую оценку через `E8` -маску.

30.2. Локальная HTTP-поверхность

Локальный сервер предоставляет:

- `GET /` — встроенный одностраничный интерфейс;
- `GET /healthz` — локальная проверка состояния и путь к архиву;
- `POST /api/is-prime` — JSON-запрос с `n` в виде строки;
- `POST /api/next-prime` — JSON-запрос с `n` в виде строки.

Большие целые передаются строками и в запросах, и в ответах, чтобы браузерный код не зависел от точности целых чисел JavaScript с плавающей точкой.

30.3. Почему интерфейс выглядит именно так

Визуальный язык следует двум ограничениям:

1. сдержанный инфраструктурный неоновый стиль вместо тяжёлого Web3-оформления;
2. геометрические мотивы из иерархии `E8 / 24-cell / тессеракт`, развитой выше в документе.

Поэтому интерфейс использует тёмное поле, тонкие решётчатые наложения и многослойные геометрические каркасные элементы, но остаётся практически локальным инструментом, а не промо-страницей.

30.4. Связь интерфейса с архивом

Интерфейс сам не выполняет подсчёт. Это тонкая локальная поверхность запросов поверх существующего исполнительного слоя и архива:

- архив остаётся источником истины;
- интерфейс выполняет только запросы на чтение;
- тот же путь на базе `wheel210` используется CLI и HTTP-слоем.

30.5. Локальный запуск

Канонический запуск:

```
make ui-start ARCHIVE_DIR=data/runs/count_1e11/archive
```

После этого открыть:

```
http://127.0.0.1:4173
```

Команды жизненного цикла:

```
make ui-status  
make ui-stop
```

Все артефакты интерфейса остаются внутри `data/`, и интерфейс соблюдает ту же дисциплину без записи в `$HOME`, что и остальные точки входа проекта.